

基于烟幕弹遮蔽的多无人机协同导弹防御优化模型

摘要

问题一，首先废除真目标被烟幕弹有效遮蔽情形的定义，即真目标完全处于以导弹为顶点且与烟幕弹球体相切的无限延伸圆锥内。为简化遮蔽判断，采用“有限元”思想将真目标圆柱体外表分割为有限个网格，通过网格中心点表征目标表面位置。再建立数学模型判断导弹与所有目标中心点连线线段是否均被烟幕弹所遮挡，遍历网格采样点搜索满足条件的 t ，最终得到有效遮蔽时间为 **1.3916s**。

问题二，为使单枚导弹有效遮蔽时间最长，优化 FY1 的速度大小 v 、速度方向角 θ 、滑翔时间 t_1 和引信时间 t_2 这四个参数。在设定参数约束范围后，采用遗传算法求解：从初始种群出发，通过评估个体适应度、执行选择（淘汰低适应度个体）、交叉（对部分个体交叉）、变异操作，不断迭代产生更优个体。最终求得最大遮蔽时长为 **4.582s**，对应最优参数为 $\theta=174.36$ 、 $v=91.7067$ 、 $t_1=0.5746$ 、 $t_2=0.6318$ 。

问题三，先确定有效遮蔽时间跨度的约束：左约束以滑翔时间与引信时间最短条件计算得出，通过将视线投影到 xoy 平面转化为追及问题推导得出右约束。令引信时间为零简化模型，遍历 θ 和 v 分析时间跨度变化趋势，确定 $v = 140m/s$ 、 $\theta = 179.85^\circ$ 。再设计三段区间最大并集算法，遍历滑翔时间与引信时间寻找有效遮蔽时间区间，最终得到三枚烟幕弹有效遮蔽时间分别为 **4.85s-7.40s**、**7.40s-10.85s**、**10.90s-12.51s**，总有效遮蔽时长 **7.61s**。

问题四：借助贪心算法，先分别优化 FY1、FY2、FY3 单架无人机烟幕弹的有效遮蔽时间，得到初步结果。为证明局部最优可组合为全局最优，将问题降维到二维，通过推导烟幕弹到达视线范围的最短时间和高度约束，证明三架无人机的遮蔽时间段互不重叠。实际仿真后，FY1、FY2、FY3 的最长遮蔽时间分别为 **3.76s-8.26s**、**14.69s-18.60s**、**26.69s-29.78s**，总时长 **11.50s**。

问题五：首先进行目标分配，借鉴追逃博弈的目标分派方案，结合视线角转率、追击点分线段比例、实际距离三个参量构建评分函数，判断出仅 FY3 和 FY4 能对 M1~M3 均进行干扰。根据任务分配，先粗略搜索满足遮盖任务的 (θ, v) ，再确定发弹命令 (t_1, t_2) ，最终实现总遮蔽时间 **36.8s**。

第1章 问题背景与重述

1、问题背景

在空军作战中，常采用投掷烟雾干扰弹的方式来遮蔽敌方导弹以保护我方目标。烟雾干扰弹控制化学物质燃烧或爆炸分散形成烟雾云团，成本低、效费比高。在使用中，需要精确控制其抛撒位置及引爆时间，以高效发挥干扰弹的屏蔽作用。

2、问题提出

一种无人机控制烟雾干扰弹的模式为：在受领任务后，由无人机挂载烟雾干扰弹进行等高度匀速飞行，经过特定巡航时间后投放烟雾干扰弹，干扰弹在重力作用下继续运动，到达起爆时间后，形成距中心范围 10M 的烟雾云团，云团保持匀速下落，为目标提供有效遮蔽。

在此情景下，3 枚敌方导弹初始位置均已知，且攻击方向指向同一坐标原点，导弹飞行速度恒为 300m/s；5 架无人机的初始位置均已知；真目标的位置固定，为半径为 7m、高 10m 的圆柱体。

我们需要解决以下问题：

问题一：考虑无人机 FY1 等高度飞行速度 120m/s、等高度飞行方向指向原点、烟雾弹投放时间点、烟雾弹引爆时间点均已知的前提下，通过无人机 FY1 投掷一枚烟雾弹对导弹 M1 进行有效遮蔽。导弹初始位置、运动速度、运动方向均确定，且初始运动速度方向与无人机速度方向在同一 y 轴坐标下。建立数学模型，计算烟雾干扰弹在起爆后的 20s 内形成的半径为 10m 的烟雾云团，对导弹 M1 提供的有效遮蔽时长。

问题二：在问题一的基础上，解除 FY1 的飞行速度、飞行方向、烟雾干扰弹的投放时间、引爆时间进行特定限制，仅满足无人机 FY1 以 70-140m/s 速度等高度飞行。建立数学模型，计算使得对 M1 有效遮蔽时间尽可能长，无人机 FY1 的最佳飞行方向、飞行速度、烟雾干扰弹的投放点和起爆点。

问题三：在问题二的基础上，无人机 FY1 可投放多枚烟雾干扰弹，建立数学模型，计算满足无人机投放两枚干扰弹的时间间隔大于 1s 的条件下，无人机 FY1 投放 3 枚烟雾干扰弹，使得对 M1 的有效遮蔽时间最长，无人机 FY1 的最佳飞行方向、飞行速度和 3 枚烟雾干扰弹各自的最佳投放点和起爆点。

问题四：在上述问题基础上，增加无人机 FY2、FY3，3 架无人机各投放一枚烟雾干扰弹对 M1 进行干扰。建立数学模型计算，使得 3 枚烟雾干扰弹对导弹 M1 进行有效遮蔽的时间总和最长，无人机 FY1、FY2、FY3 投放烟雾弹的投放策略。

问题五：利用 5 架已知初始位置的无人机，每架无人机最多投放 3 枚烟雾弹，对 M1、M2、M3 三枚导弹进行干扰，满足任意烟雾干扰弹对任意导弹进行了有效遮蔽的时间点，即属于有效遮蔽时长，遮蔽时长可以间断。建立数学模型，计算使得有效遮蔽时间总和最长的烟雾弹投放策略。

第2章 问题分析

对问题一，首先需建立导弹、无人机、烟幕弹及目标的空间几何关系。已知FY1 和导弹M1 的速度大小和方向，构建一个运动模型。利用有限元思想，将真目标表面离散化为网格点，并逐一检测这些点与导弹之间的视线是否穿过烟幕球体，从而确定目标是否处于遮蔽状态。进一步，通过遍历导弹飞行时间序列，记录有效遮挡区间的起止时刻，即可得出总的有效遮蔽时长。^{[2]†}

针对问题二，建立一个优化模型，求解在不确定FY1 飞行方向、速度大小以及烟雾干扰弹投放和爆炸点情况下，烟雾干扰弹对真目标的最长遮蔽时间。由题目初始条件，我们可以初步确定FY1 飞行方向角和速度大小的范围。由于决策范围较大，且目标函数非线性、不规则，适合遗传算法优化。遗传算法通过选择、交叉、变异迭代更新候选解，逐步逼近全局最优解。在粗略搜索后，逐步缩小时间步长，进一步细致搜索，从而得到精度较高的最大遮蔽时长。

针对问题三，我们统一三枚烟雾弹的运动学计算路径，再寻找三段区间覆盖并集最大值，即可得到问题三所求烟雾遮蔽最大值。计算一枚烟雾弹不同投射时间的遮蔽最大值，我们通过逐步缩小范围确定最佳FY1 飞行速度与飞行方向，再遍历搜索合理范围内的飞行时间与投放时间，计算烟雾弹不同投射时间的遮蔽时间，进而寻找三段遮蔽时间并集最大值，即可得到最佳投放策略。

针对问题四，要求三架无人机FY1、FY2、FY3 协同，各投放一枚烟幕弹，以延长对M1 的遮蔽时长。需判断三架无人机的遮蔽区间是否存在重叠或间隙。若区间互不重叠，则总时长为三者之和；由于三架无人机位置不同，进入导弹视线的时间有所差异，可计算出遮蔽时间并不重合，可通过局部最优组合近似实现全局最优。将每架无人机的投放优化问题单独求解，得到各自可实现的最长遮蔽时长区间。再在区间上进行组合，选择非重叠区间。最终输出三机的投放方案。

针对问题五，问题扩展为五架无人机协同，每一架无人机最多投放三枚烟雾干扰弹，

† 本章节部分使用 AI，见参考文献

对三枚导弹M1、M2、M3 进行遮蔽。此时问题规模更大，需先进行任务分配：结合无人机与导弹的相对位置、速度约束，可以推断部分无人机难以干扰所有导弹，从而可缩小任务范围。分配后，再对每个无人机采用问题三、四中的优化方法，确定烟幕弹投放策略。最后整合所有结果，得到总体最优遮蔽方案。

第3章 模型假设

1. 忽略空气阻力对于烟幕弹未爆炸前的运动的影响；
2. 烟幕弹在下落时，始终保持竖直方向、并且匀速；
3. 只考虑飞机与飞机之间的协同，不考虑烟幕弹之间的协同作用；
4. 考虑节省烟幕弹，尽可能减少烟幕弹投弹量。

第4章 本文所涉及的符号

G	烟幕弹球心/烟幕弹质心
M_i	i 号导弹位置
P	圆柱体表面采样点/网格
O	假目标位置/原点
v	飞行速度
θ	飞行方向角
t_1	滑翔时间
t_2	引信时间

第5章 模型建立与求解

1、问题一模型的建立与求解

首先，本模型定义真实目标被烟幕弹遮蔽的情形。本文称第*i*个导弹的实时位置为 M_i ，无人机所投掷的烟幕弹实时位置为 G ，从导弹的视角出发，烟雾云团球体 G 相对于 M_1 形成的视野盲区，是以 M_1 点为顶点，以 M_1 点和 G 连线为高，外表面与球体 G 相切的无限延伸的圆锥。当真目标完全位于该圆锥内时，定义真目标被烟幕弹遮蔽。

对真目标的遮蔽进行几何上完全解析的判断相对困难，本文通过“有限元”思想以网格形式分割真目标圆柱体的外表面，用网格的中心点集体表征其附近区域的位置信息，将连续的目标表面离散化，如图 5.1 所示。在本文的求解过程中，上表面划分 448 个网

格，侧面划分 640 个网格，总计 1086 个。

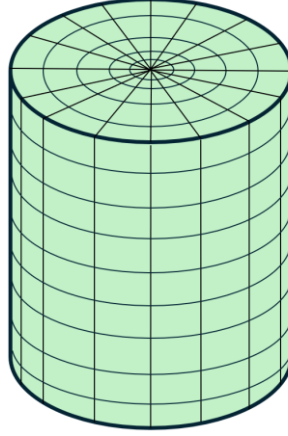


图 5.1 真实目标表面网格划分示意图

如图 5.2 所示，设所计算的网格中心为 P 点，所有 P 点与导弹连线线段 PM 构成集合 C ，即为导弹对真目标的全部视线。当全部视线均被烟幕弹遮挡时，即为有效遮蔽的数学条件。若存在某个视线 PM 和球体 G 无交点或交点不在线段 PM 上，此时可以判定遮蔽无效。据此建立有效遮蔽的数学模型：

$$K = P + k(M - P) \quad (1)$$

$$\exists k \in (0,1), \text{s.t.} \|KG\| \leq 10 \quad (2)$$

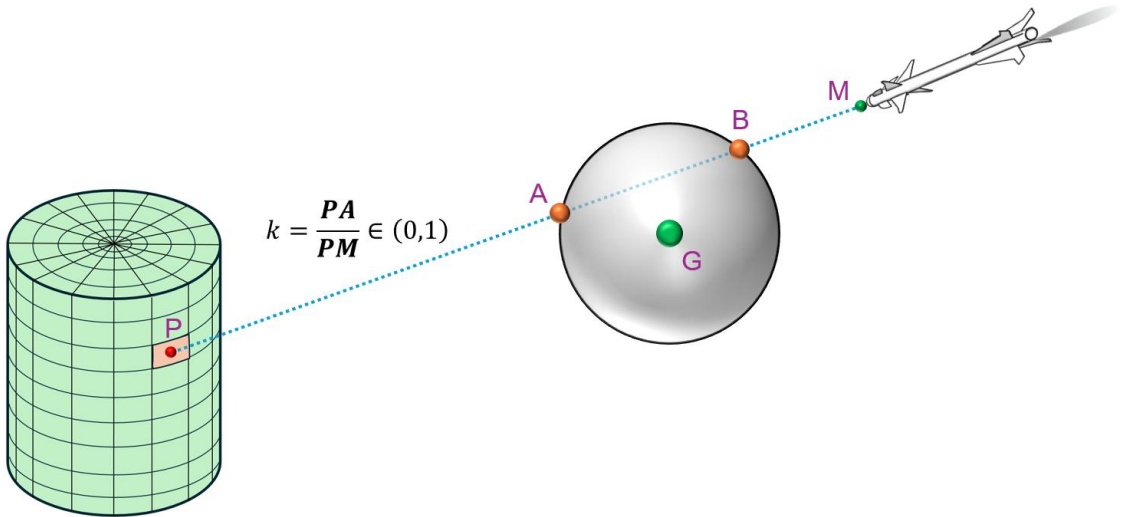


图 5.2 有效遮蔽几何模型示意图（比例经过调整）

计算经过时间 t 后的 M 、 G 点坐标，遍历所有网格采样点 P ，搜索满足条件的 t ，计算得到有效遮蔽时间 Δt 为1.3916s。

2、问题二模型的建立与求解

定义 FY1 速度大小为 v ，以 x 轴正半轴为始边，速度方向角为 θ ，滑翔时间和引信时间分别为 t_1 和 t_2 ，我们采用遗传算法对 $X = (\theta, v, t_1, t_2)^T$ 进行优化，使得根据问题一模型

定义的总的有效遮蔽时间 T 最大。

遗传算法模拟自然界生物遗传进化，从任一初始种群出发，通过选择、变异和交叉三种算子，对个体对象进行操作，产生更适合环境的个体。最后不断逼近至指定问题最优解，一种实现方式可参看表 5.1 所示的算法

表 5.1 遗传算法伪代码

算法 1：遗传算法	
输入： 列向量 $\mathbf{X}$	
1:	初始化种群 $P(0), t=0$;
2:	while $t \leq T$ do :
3:	for $i=1$ to M do :
4:	评估种群 $P(t)$ 的适应度
5:	end for
6:	for $i=1$ to M do :
7:	对种群中的个体执行选择操作
8:	end for
9:	for $i=1$ to $M/2$ do :
10:	对种群中的个体执行交叉操作
11:	end for
12:	for $i=1$ to M do :
13:	对种群中的个体执行变异操作
14:	end for
15:	for $i=1$ to M do :
16:	将操作后的个体更新到下一代种群
17:	$P(t+1)=P(t)$
18:	end for
19:	$t=t+1$
20:	end while
输出： 目标遮挡时长	

如图 5.3 所示，遗传算法通过对优化问题进行编码得到染色体，而选择、变异和交叉均基于染色体进行。如所示，通过给定一种评价体系，算法将淘汰掉评分较低的个体，更多地保留和优化评分较好的个体，从而向全局最优收敛。

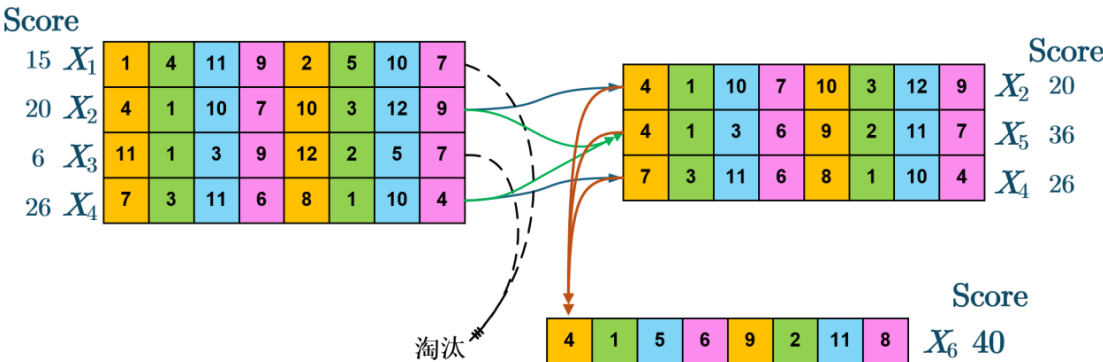


图 5.3 一种遗传算法图解

为了使烟幕弹能够接近视线 PM ，飞行方向 θ 应接近 π ，按照以下数据设定 $\mathbf{X} = (\theta, v, t_1, t_2)^\top$:

$$\begin{cases} \theta \in \left(\frac{7\pi}{8}, \pi\right) \\ v \in (70, 140) \\ t_1 \in (0, 20) \\ t_2 \in (0, 20) \end{cases} \quad (3)$$

通过遗传算法由粗略到细致进行优化搜索，求得最大遮蔽时长为4.582s，其中 $\theta = 174.36^\circ, v = 91.7067, t_1 = 0.5746, t_2 = 0.6318$ 。

3、问题三模型的建立与求解

注意到当 $t_1 = t_2 = 0$ 时， θ, v 对烟雾弹投掷位置不产生影响，进一步的验证表明，该点是无人机 FY1 投掷烟雾弹后能够产生有效遮蔽的全部可能时间中最早的一点，计算表明该时间为4.85s，此即时间跨度的左约束。除此之外，烟雾弹还存在时间跨度的右约束，考虑无人机携带烟幕弹接近导弹的视线 PM ，将 PM 投影到 xoy 平面，这一追及问题便降维为如下关系：

$$20000 - \frac{3000}{\sqrt{101}}t = 17800 - vt \cos \theta \quad (4)$$

进而得到：

$$t = \frac{2200}{\frac{3000}{\sqrt{101}} - v \cos \theta} \leq 14 \quad (5)$$

为了使得三枚烟幕弹接力遮挡的总时间达到最长，应当先寻找 θ, v ，使得在发弹参数 (t_1, t_2) 控制下，FY1 投掷烟雾弹后能够产生有效遮蔽的全部可能时间的跨度最大。对该模型进行简化，令引信时间 $t_2 = 0$ ，烟幕弹投出后即爆炸，在此假设下遍历 θ, v ，如表 5.2 所示。

表 5.2 一些特殊角度-速度组合下时间跨度表

θ	v	ΔT	θ	v	ΔT	θ	v	ΔT
0	70	4.66	0.02	140	8.88	0.3	140	8.85
0	100	6.22	0.05	130	7.99	0.7	140	5.57
0	130	7.7	0.05	135	8.65	0.5	140	6.5
0	140	8.88	0.05	140	8.88	0.4	140	7.5
0.1	130	7.98	0.15	140	8.87	0.45	140	6.95
0.1	140	8.88	1	140	5.25	1	140	5.24

时间跨度 ΔT 随着 v 变大而减小，随着 θ 变大先变大后变小，整体呈现出如图 5.4 所示趋势。

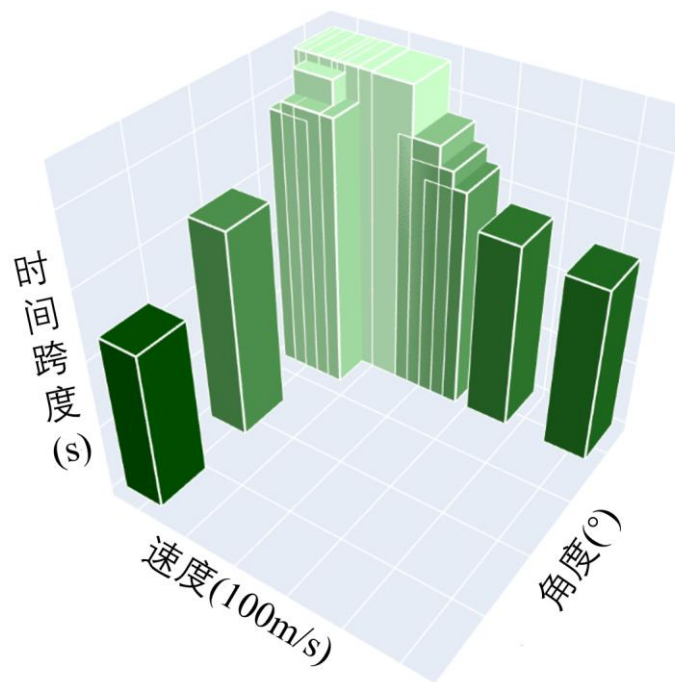


图 5.4 $v - \theta - \Delta T$ 三方图

进一步细化搜索，确定 $v = 140m/s, \theta = 179.85$ ，以此求解最佳滑翔时间 t_1 与引信时间 t_2 ，通过遍历所有可能的发弹命令 (t_1, t_2) ，寻找所有可能的有效遮蔽时间范围，如图 5.5 所示。这些时间范围存在重叠，本文设计如表 5.3 所示的三段区间最大并集算法，找到三段最大有效遮蔽时间段之和，得到最大有效遮蔽时长，并分别记录此三段数据的滑翔时间 t_1 与引信时间 t_2 ，即为所求最佳投放策略点。

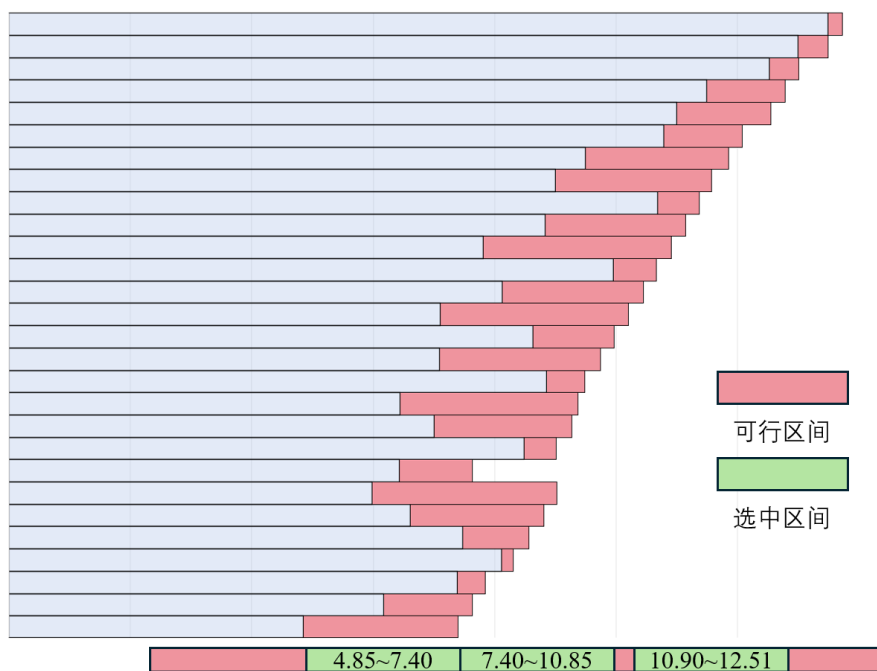


图 5.5 可行区域内部搜索结果图

表 5.3 三段区间最大并集求解伪代码

算法 2：三段区间最大并集

输入： 区间列表

```
1: 遍历区间记录最大右端点 mx
2: 初始化 end path 列表;
3: 遍历区间 end 更新为包含下标的所有区间中最小的左端点;
4: for i in range(3) do
5:     初始化 dp_c 列表,记录当前时间点的最大区间长度
6:     初始化 path_c 列表,记录选择区间
7:     for i in range(1,mx+1) do
8:         更新 dp_c[i]
9:         更新 path_c[i]
10:    end for
11:    dp=dp_c
12:    path=path_c
13: end for
14: 辅助函数计算区间并集长度
```

输出： 区间以及总长度结果

计算得到第一枚烟雾干扰弹的有效遮蔽时间为 $4.85s - 7.40s$ ，第二枚有效遮蔽时间为 $7.40s - 10.85s$ ，第三枚有效遮蔽时间为 $10.90s - 12.51s$ ，总体有效遮蔽时长为 $7.61s$ 。无人机 FY1 投放三枚烟雾弹的最佳投放策略见表格文件 result1.xlsx。

4、问题四模型的建立与求解

三个无人机进行协同耦合计算相对困难，为此，本文借助贪心算法，先优化每一架无人机所发射烟幕弹的有效遮蔽时间，再证明这是全局最优的。

借助问题 2 的算法，使用全局搜索和局部启发搜索相结合的方式，得到了 FY1 的最长遮蔽时间为 $4.5s$ ，FY2 的最长遮蔽时间为 $3.91s$ ，FY3 的最长遮蔽时间为 $3.09s$ 。

下面证明这是全局最优解：

命题：FY1, FY2, FY3 三者遮蔽时间段相互分散而互不重叠，进而上述局部最优可以组合成全局最优

为了简化计算，假设目标 P 是一个无形质点，烟幕弹 G 需处在导弹 M 与目标 P 连线中间位置，才可提供有效遮蔽。将此问题降阶到二维，如图 5.6 所示，FY3 所投烟幕弹至少需要飞行到视线扫过范围，才可能有效遮挡导弹视线，因此有：

$$t_{i,min} = \frac{d_i}{v_{max}} \quad (6)$$

进一步考虑最大的情形，在高度上有：

$$20000 - \frac{300}{\sqrt{101}} t_{i,max} \geq z_i \quad (7)$$

根据以上约束计算得到，FY2 离开可行域时 FY3 仍未进入可行域，这也就证明了三者遮蔽时间段相分离，互不重叠，因此是全局最优的。

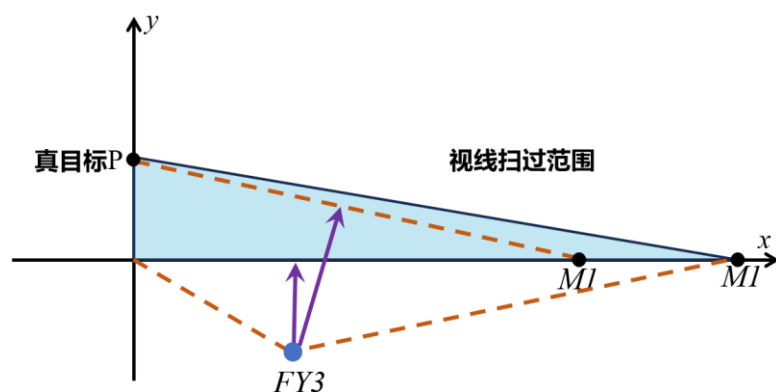


图 5.6 无人机飞行时间约束估算示意图

实际仿真得到 FY1 的最长遮蔽时间为 $3.76s - 8.26s$ ，FY2 的最长遮蔽时间为 $14.69s - 18.60s$ ，FY3 的最长遮蔽时间为 $26.69s - 29.78s$ ，最长时间总和为 $11.50s$ 。三架无人机 FY1、FY2、FY3 投放干扰弹的投放策略见表格文件 result2.xlsx。

5、问题五模型的建立与求解

首先进行目标分配。根据题中所给的速度数据和无人机位置信息，结合问题 4 所运用的约束，可以预见到部分无人机无法完成对 3 枚导弹的接力遮挡。如 FY1 只能完成对 M1 的遮挡，FY2 和 FY5 只能完成其同一侧导弹的遮挡。

Dong 等于 2021 年提出了一种追逃博弈问题的目标分派方案^[1]，通过一些参数的评估，通过补偿拍卖算法加以分配。本文借助这种思想，将烟幕弹垂直到达该时刻下视线的位置定义为追及目标，并结合目标视线角转率、追击点分线段比例和实际距离三个参量加以评估，如图 5.7 所示。

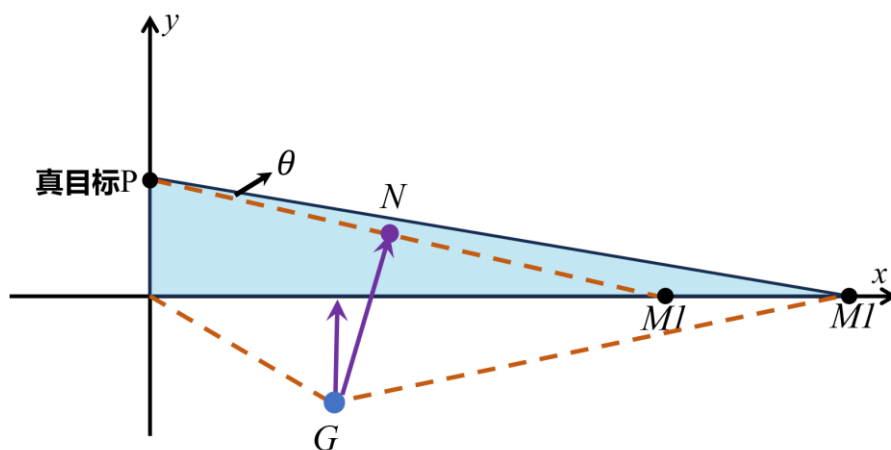


图 5.7 各追及运动参数示意

定义目标视线角转率为：

$$\beta = \frac{d\theta}{dt} \quad (8)$$

定义追击点分线段比例为：

$$\lambda = \frac{NM}{PM} \quad (9)$$

特别的，当 $\lambda > 1$ 或 $\lambda < 0$ 时，该部分得分为 0

整体的评分函数为：

$$P = P_1 + P_2 + P_3 = e^{\frac{1}{\beta}} + e^{\frac{1}{\lambda}} + e^{\min(GN-10,0)} \quad (10)$$

通过对各个飞行器进行评分，可以判断出只有 FY3 和 FY4 能够对 M1~3 均进行干扰，其余飞行器均出现了快速收敛下降的局面，说明可能对部分导弹无法形成干扰

结合以上的判断，可以得到各个飞机所投烟幕弹可以覆盖的导弹。根据这个任务分配，先粗略搜索能够满足对应遮盖任务的 (θ, v) ，再确定地搜索对应的发弹命令 (t_1, t_2) ，最后得到以下结果，总时间达到36.8s，如图 5.8，其余具体参数见支撑材料。

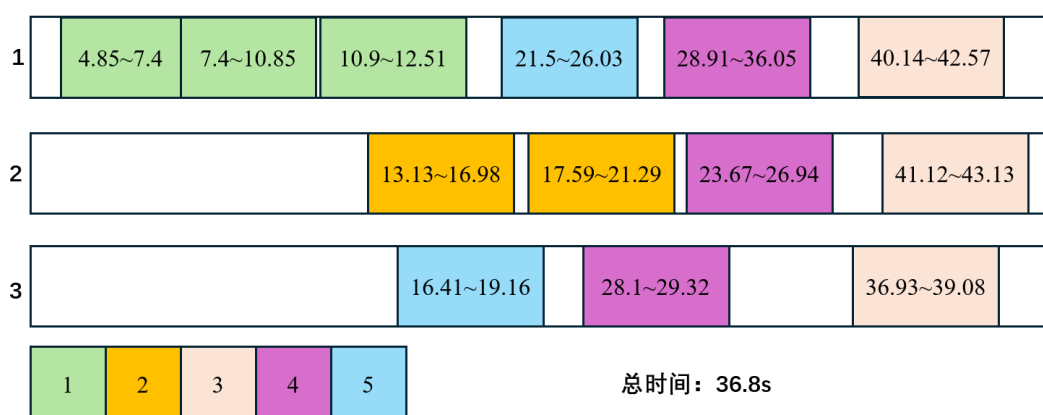


图 5.8 协同遮蔽结果示意图

整体上看，对 M1 的遮蔽最为有利，这是由 FY1 特殊的位置关系决定的，排除这一因素后。M2 的遮蔽效果最好，这和 M2 的高度最高有关，导弹高度越高，留给烟幕弹遮蔽的机会更长，模型结果是符合常理的。

第6章 模型分析与评价

1、模型优点

我们的模型具有明确的几何意义，在能够快速缩小寻找范围，相比于全局使用启发式搜索，大幅度地节省了算力，并且全局搜索有利于提升全局寻解的可能性，避免了陷入局部最优

我们的模型创新性良好。第三问至第五问，我们利用动态规划思想，通过动态规划方法对时间进行统计。第五问中，我们设计了一个创新的评分函数，融合考虑了角度变化率，位置比例和距离约束条件。

2、模型缺点

受到算力和时间制约，我们的模型计算结果的精度有待提升，对待一些可能出现的最优解缺乏进一步的判断。

参考文献

- [1] QINGZE L, XIWANG D, QINGDONG L, et al. Weapon Target Assignment Strategy for Multi-Missile Cooperative Guidance with Auction Algorithm[C]//2021 40th Chinese Control Conference (CCC). Shanghai: IEEE, 2021: 3715-3720.
- [2] 豆包, Doubao-Seed-1.6, 字节跳动, 2025-09-07

支撑材料目录

1. 题解部分:

Result 1.xlsx

Result 2.xlsx

Result 3.xlsx

2. 源代码部分:

总原型库 *supple.py*

题解源代码 *P1.py* *P2.m* *P3-P4-P5.py*

寻并集代码: *findlength.py*

3. 全局搜索记录 *搜索记录.xlsx*

4. 遗传算法图 *ga2.fig*

5. AI 使用

源代码

```
import math
import numpy as np

class sol:
    """输入导弹编号和飞机坐标"""
    def __init__(self,num,GRD):
        self.num = num
        self.GG1,self.GG2,self.GG3=GRD

    def s2e(self,a, b, c):
        if a == 0 or b**2 - 4*a*c < 0:
            return None
```

```

else:
    delta = math.sqrt(b**2 - 4*a*c)
    x1 = (-b + delta) / (2*a)
    x2 = (-b - delta) / (2*a)
    return x1, x2

def dt(self, th, v, t1, t2):
    # 常数
    g = 9.8
    alpha = 3000 / math.sqrt(101)
    beta = 300 / math.sqrt(101)

    alpha2=300*(190/math.sqrt(190**2+6**2+21**2))
    beta2= 300*(21/math.sqrt(190**2+6**2+21**2))
    gamma2= 300*(6/math.sqrt(190**2+6**2+21**2))

    alpha3=300*(180/math.sqrt(180**2+6**2+19**2))
    beta3=300*(19/math.sqrt(180**2+6**2+19**2))
    gamma3=-300*(6/math.sqrt(180**2+6**2+19**2))

    if self.num==1:M = (20000 - (t1 + t2) * alpha, 0, 2000 - (t1 + t2) *
beta)
    elif self.num==2: M = (19000 - (t1 + t2) * alpha2, 600-(t1+t2)*gamma2,
2100 - (t1 + t2) * beta2)
    else: M = (18000 - (t1 + t2) * alpha3, -600-(t1+t2)*gamma3, 1900 - (t1
+ t2) * beta3)

    GRD = (self.GG1 - v * (t1 + t2) * math.cos(th),
self.GG2 + v * (t1 + t2) * math.sin(th),
self.GG3- 0.5 * g * t2 * t2)

    # 画网格
    mesh = []
    for theta in np.arange(0, 2 * math.pi, 0.2*math.pi):
        for r in np.arange(0, 7, 0.5):
            mesh.append((r * math.cos(theta), 200 + r * math.sin(theta),
10))
    for theta in np.arange(0, 2 * math.pi, 0.2*math.pi):
        for z in np.arange(0, 10, 0.5):
            mesh.append((7 * math.cos(theta), 200 + 7 * math.sin(theta),
z))

    # 检测函数
    def solvek(t):
        ifclose = False

```

```

        if self.num==1:M1, M2, M3 = (M[0] - alpha* t, M[1], M[2] - beta*
t)#1
        elif self.num==2: M1, M2, M3 = (M[0] - alpha2* t, M[1]-gamma2*t,
M[2] - beta2* t)#2
        elif self.num==3: M1, M2, M3 = (M[0] - alpha3* t, M[1]-gamma3*t,
M[2] - beta3* t)#3
        G1, G2, G3 = (GRD[0], GRD[1], GRD[2] - 3 * t)
        for point in mesh:
            P1, P2, P3 = point
            a = (M1 - P1)**2 + (M2 - P2)** 2 + (M3 - P3)**2
            b = (-2) * ((M1 - P1) * (G1 - P1) + (M2 - P2) * (G2 - P2) + (M3
- P3) * (G3 - P3))
            c = (G1 - P1)** 2 + (G2 - P2)**2 + (G3 - P3)** 2 - 10**2
            sol = self.s2e(a, b, c)
            if sol is not None and 0 <= sol[1] <= 1:
                ifclose = True
            else:
                return (t, False)
        return (t, ifclose)

# 时间遍历
cntt = 0
t_start = 0
t_end = 20
t_step = 0.01
cnttable=[]
for t in np.arange(t_start, t_end, t_step):

    if solvek(t)[1]:
        cntt += 1
        cnttable.append(t)

    if cnttable :return (cntt - 1) *
t_step,float(cnttable[0]),float(cnttable[-1])
    else:return None

def run(self):
    for t1 in np.arange(11, 13, 0.1):
        for t2 in np.arange(12,14,0.1):
            # for theta in np.arange(-135*np.pi/180,-
45*np.pi/180,1*np.pi/180):
            #for theta in np.arange(-45*np.pi/180,-135*np.pi/180,-
2*np.pi/180):

                #for v in np.arange(85, 95, 1):
                for v in np.arange(95,105,1):
                    #print(theta,v,t1,t2)

```

```

# v=90

theta=-53.0*np.pi/180
if self.dt(theta, v, t1, t2) is not None:
    print(float(theta*180/np.pi), v, t1,
t2,self.dt(theta, v, t1, t2))

```

```

import math
import numpy as np
import supple

sol=supple.sol(1,(17800,0,1800))

if sol.dt(0,120,1.5,3.6) is not None:
    print(sol.dt(0,120,1.5,3.6))[0]
else:
    print("无解")

```

```

%% GA 优化部分
clear; clc; close all;

% 优化参数配置
nVars = 4;
lb = [pi-0.1, 70, 0, 0];
ub = [pi+0.1, 100, 1, 1];

% GA 设置
options = optimoptions('ga', ...
    'PopulationSize', 100, ...
    'MaxGenerations', 15, ...
    'CrossoverFraction', 0.6, ...
    'MutationFcn', @mutationadaptfeasible, ...
    'Display', 'iter', ...
    'PlotFcn', @gaplotbestf);

[x_opt, fval, exitflag, output] = ga(@ga_fitness, nVars, [], [], [], [], lb,
ub, [], intcon, options);
dt_opt = -fval;

fprintf('===== 遗传算法优化结果 =====\n');
fprintf('最优参数组合: \n');
fprintf('  角度 th = %.4f 弧度 (%.2f 度)\n', x_opt(1), rad2deg(x_opt(1)));
fprintf('  速度 v = %.4f\n', x_opt(2));
fprintf('  时间 t1 = %.4f\n', x_opt(3));
fprintf('  时间 t2 = %.4f\n', x_opt(4));

```

```

fprintf('最优参数对应的 dt 值: %.4f\n', dt_opt);

%% 原型函数 (同 supple)
function sol = s2e(a, b, c)
    if a == 0 || (b^2 - 4*a*c) < 0
        sol = [];
    else
        delta = sqrt(b^2 - 4*a*c);
        x1 = (-b + delta) / (2*a);
        x2 = (-b - delta) / (2*a);
        sol = [x1, x2];
    end
end

function dt_val = dt(th, v, t1, t2)
    g = 9.8;
    alpha = 3000 / sqrt(101);
    beta = 300 / sqrt(101);

    M = [20000 - (t1 + t2)*alpha, 0, 2000 - (t1 + t2)*beta];
    GRD = [17800 - v*(t1 + t2)*cos(th), ...
           v*(t1 + t2)*sin(th), ...
           1800 - 0.5*g*t2^2];

    % 生成网格点
    mesh = [];
    for theta = 0:0.1:2*pi - 1e-6
        for r = 0:0.1:7
            x = r * cos(theta);
            y = 200 + r * sin(theta);
            z = 10;
            mesh = [mesh; x, y, z];
        end
    end
    for theta = 0:0.1:2*pi - 1e-6
        for z = 0:0.1:10
            x = 7 * cos(theta);
            y = 200 + 7 * sin(theta);
            mesh = [mesh; x, y, z];
        end
    end

    function [t, ifclose] = solvek(t)
        ifclose = false;

```



```

M_new = [M(1) - alpha*t, 0, M(3) - beta*t];
G_new = [GRD(1), GRD(2), GRD(3) - 3*t];

for i = 1:size(mesh, 1)
    point = mesh(i, :);
    P1 = point(1); P2 = point(2); P3 = point(3);
    M1 = M_new(1); M2 = M_new(2); M3 = M_new(3);
    G1 = G_new(1); G2 = G_new(2); G3 = G_new(3);

    a = (M1 - P1)^2 + (M2 - P2)^2 + (M3 - P3)^2;
    b = -2 * ((M1 - P1)*(G1 - P1) + (M2 - P2)*(G2 - P2) + (M3 - P3)*(G3
- P3));
    c = (G1 - P1)^2 + (G2 - P2)^2 + (G3 - P3)^2 - 10^2;

    sol = s2e(a, b, c);
    if ~isempty(sol) && any(sol >= 0 - 1e-6) && any(sol <= 1 + 1e-6)
        ifclose = true;
    else
        ifclose = false;
        break;
    end
end
end

cntt = 0;
t_start = 0;
t_end = 10;
t_step = 0.002;
t_list = t_start:t_step:t_end;

for t = t_list
    [~, ifclose] = solvek(t);
    if ifclose
        cntt = cntt + 1;
    end
end

dt_val = (cntt - 1) * t_step;
end

function fitness = ga_fitness(x)
    th = x(1);
    v = x(2);
    t1 = x(3);
    t2 = x(4);

```

```

    dt_val = dt(th, v, t1, t2);
    fitness = -dt_val;
end

```

```

import supple
import numpy as np

sov=supple.sol(1,(17800,0,1800)) # 这是一个实例
for t1 in np.arange(0, 60, 1):#粗搜
    for t2 in np.arange(0,10,1):
        for theta in np.arange(0*np.pi/180,10*np.pi/180,1*np.pi/180):
            for v in np.arange(70,140,10):
                if sov.dt(theta, v, t1, t2) is not None:
                    print(float(theta*180/np.pi), v, t1, t2,sov.dt(theta, v, t1,
t2))

for t1 in np.arange(0, 8, 0.1):#细搜
    for t2 in np.arange(0,10,0.1):
        for theta in np.arange(0*np.pi/180,1*np.pi/180,0.1*np.pi/180):
            for v in np.arange(135,140,0.1):
                if sov.dt(theta, v, t1, t2) is not None:
                    print(float(theta*180/np.pi), v, t1, t2,sov.dt(theta, v, t1,
t2))

```

```

import pandas as pd

# 读取 CSV 专用
# df = pd.read_csv('data.csv')

# # 读取 Excel 文件
# df = pd.read_excel("FY2import3.xlsx", sheet_name="Sheet3", usecols="H:I",
header=None)
#
# # 转换成二维列表 [[x1, y1], [x2, y2], ...]
# intervals = df.values.tolist()

# 如果是空格分隔
#intervals = pd.read_csv("data.xlsx", sep='\s+', header=None)
# 测试用例
intervals = [
    [11.8, 12.07], [11.8, 12.41], [11.8, 12.67], [11.8, 12.73], [11.8, 12.85],
    [11.8, 12.92],

```

```

    [11.9, 12.09], [11.9, 13.06], [11.9, 13.65], [11.9, 14.13], [11.9, 14.52],
    [11.9, 14.83],
    [11.9, 15.07], [11.9, 15.25], [11.98, 15.37], [12.38, 15.43], [12.64,
    15.93], [12.66, 15.39],
    [12.81, 15.37], [13.05, 14.46], [13.13, 16.98], [13.35, 17.2], [13.44,
    15.17], [13.59, 17.37],
    [13.84, 17.51], [14.09, 17.6], [14.34, 17.67], [14.6, 17.7], [14.86,
    17.68], [15.01, 18.31],
    [15.09, 18.62], [15.12, 17.63], [15.15, 17.41], [15.2, 18.87], [15.32,
    19.07], [15.41, 17.54],
    [15.46, 19.24], [15.6, 19.37], [15.74, 17.42], [15.75, 16.37], [15.75,
    19.48], [15.89, 19.56],
    [16.02, 19.62], [16.11, 17.22], [16.15, 19.67], [16.27, 19.7], [16.37,
    19.73], [16.46, 19.74],
    [16.53, 19.75], [16.58, 19.76], [17.14, 20.27], [17.17, 20.5], [17.18,
    19.98], [17.21, 20.68],
    [17.27, 20.84], [17.28, 19.61], [17.33, 20.96], [17.39, 21.06], [17.45,
    21.14], [17.51, 21.2],
    [17.56, 19.06], [17.56, 21.25], [17.59, 21.29], [19.12, 21.93], [19.12,
    21.93], [19.12, 21.93],
    [19.14, 21.92], [19.18, 21.81], [19.25, 21.59], [19.38, 21.29], [19.65,
    20.83], [21.57, 21.91]
]
# 如果需要乘 100 并四舍五入
intervals = [[round(x*100), round(y*100)] for x, y in intervals]

# k = 3
# #intervals=[[1,3],[2,4],[5,7],[8,11],[9,13],[15,17]]
# mx = max([i[1] for i in intervals])
# end, dp = [mx + 10] * (mx + 1), [0] * (mx + 1)
# path = [[] for _ in range(mx + 1)]
#
# for idx, (s, e) in enumerate(intervals):
#     for j in range(s, e + 1):
#         if j <= mx: # 防止越界
#             end[j] = min(end[j], s)
#
# for _ in range(k):
#     dp_c = [0] * (mx + 1)
#     path_c = [[] for _ in range(mx + 1)]
#
#     for i in range(1, mx + 1):
#         x = end[i]
#         dp_c[i] = dp_c[i - 1]
#         path_c[i] = path_c[i - 1].copy()
#

```

```

#         length = i - x
#         if 1 <= x <= mx and dp[x - 1] + length > dp_c[i]:
#             dp_c[i] = dp[x - 1] + length
#             path_c[i] = path[x - 1].copy()
#             # 找对应区间索引
#             for idx2, (s2, e2) in enumerate(intervals):
#                 if s2 <= x and e2 >= i:
#                     path_c[i].append(idx2)
#                     break
#             elif x <= 0 or x > mx:
#                 continue # 跳过不合法的 x
#
#     dp = dp_c.copy()
#     path = path_c.copy()
#
# def calcu(x1,x2,x3):
#     if x1[1]>=x2[0]:
#         if x2[1]>=x3[0]:
#             return x3[1]-x1[0]
#         else:
#             return x3[1]-x3[0]+x2[1]-x1[0]
#     else:
#         if x2[1]>=x3[0]:
#             return x1[1]-x1[0]+x3[1]-x2[0]
#         else:
#             return x1[1]+x2[1]+x3[1]-x1[0]-x2[0]-x3[0]
#
# print("选中的区间: ", [intervals[i] for i in path[-1]])
# x1, x2, x3 = [intervals[i] for i in path[-1]]
# res=calcu(x1,x2,x3)
# print(res)

k = 2
#intervals=[[1,3],[2,4],[5,7],[8,11],[9,13],[15,17]]
mx = max([i[1] for i in intervals])
end, dp = [mx + 10] * (mx + 1), [0] * (mx + 1)
path = [[] for _ in range(mx + 1)]

for idx, (s, e) in enumerate(intervals):
    for j in range(s, e + 1):
        if j <= mx: # 防止越界
            end[j] = min(end[j], s)

for _ in range(k):
    dp_cur = [0] * (mx + 1)
    path_cur = [[] for _ in range(mx + 1)]

```

```

for i in range(1, mx + 1):
    x = end[i]
    dp_cur[i] = dp_cur[i - 1]
    path_cur[i] = path_cur[i - 1].copy()

    length = i - x
    if 1 <= x <= mx and dp[x - 1] + length > dp_cur[i]:
        dp_cur[i] = dp[x - 1] + length
        path_cur[i] = path[x - 1].copy()
        # 找对应区间索引
        for idx2, (s2, e2) in enumerate(intervals):
            if s2 <= x and e2 >= i:
                path_cur[i].append(idx2)
                break
    elif x <= 0 or x > mx:
        continue # 跳过不合法的 x

dp = dp_cur.copy()
path = path_cur.copy()

def calcu(x1,x2):
    if x1[1]>=x2[0]:
        return x2[1]-x1[0]
    else:
        return x1[1]+x2[1]-x1[0]-x2[0]

print("选中的区间: ", [intervals[i] for i in path[-1]])
x1, x2= [intervals[i] for i in path[-1]]
res=calcu(x1,x2)
print(res)

```